

SQLAlchemy 2.0 & HTTP Methods Guide

Detailed Operational Mechanics, Deep Concepts & Real-World API Patterns

Core Concept in 2.0: SQLAlchemy 2.0 removes the old `session.query()` style in favor of explicit SQL construct objects (`select`, `insert`, `update`, `delete`) executed directly through `session.execute()` or `session.scalars()`.

0. BASE SETUP & DOMAIN MODEL

SETUP

```
MODELS.PY – FULLY TYPED DECLARATIVE BASE
from typing import Optional
from sqlalchemy import String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(50), unique=True)
    email: Mapped[str] = mapped_column(String(100))
    bio: Mapped[Optional[str]] = mapped_column(String(255), default=None)
    is_active: Mapped[bool] = mapped_column(default=True)
```

1. HTTP GET → RETRIEVE DATA (SELECT)

GET

HOW GET WORKS IN SQLALCHEMY 2.0

HTTP GET is **read-only and idempotent**. In SQLAlchemy 2.0:

- **By Primary Key:** Use `session.get(Model, pk)`. It first checks session cache (Identity Map); if present, it avoids hitting the database altogether.
- **By Filter Query:** Build a statement with `select(User).where(...)` and pass it to `session.scalars()` to unpack 1D ORM object instances.

```
GET ENDPOINT IMPLEMENTATION (FASTAPI EXAMPLE)
# --- Single Resource Lookup by ID ---
async def get_user_by_id(user_id: int, session: AsyncSession):
    # session.get() yields User directly or None if missing
    user = await session.get(User, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return user

# --- Filtered Collection with Pagination & Ordering ---
async def list_active_users(limit: int, offset: int, session: AsyncSession):
    stmt = (
        select(User)
        .where(User.is_active == True)
        .order_by(User.id)
        .offset(offset)
        .limit(limit)
    )
    result = await session.scalars(stmt)
    return result.all() # Sequence[User]
```

2. HTTP POST → CREATE DATA (INSERT)

POST

HOW POST WORKS IN SQLALCHEMY 2.0

HTTP POST is **non-idempotent** and creates new entities.

- **ORM Instance Method:** Instantiate `User(...)`, call `session.add()`, and commit. The entity enters the *Pending* state and transitions to *Persistent* upon commit.
- **Refresh Step:** Calling `await session.refresh(obj)` re-reads auto-generated values like primary keys or database defaults from DB.

POST IMPLEMENTATION – IDIOMATIC CREATION & HIGH-SPEED CORE

```
# --- Standard ORM Pattern (Recommended for Web APIs) ---
async def create_user(payload: UserCreateSchema, session: AsyncSession):
    new_user = User(
        username=payload.username,
        email=payload.email,
        bio=payload.bio
    )
    session.add(new_user)
    await session.commit()
    await session.refresh(new_user) # Populates auto-generated ID
    return new_user

# --- Bulk Core Insert Pattern (High Throughput) ---
async def bulk_insert_users(data: list[dict], session: AsyncSession):
    stmt = insert(User).returning(User)
    result = await session.execute(stmt, data) # Pass list of dicts
    await session.commit()
    return result.scalars().all()
```

3. HTTP PUT → COMPLETE REPLACEMENT (UPDATE)

PUT

HOW PUT WORKS IN SQLALCHEMY 2.0

HTTP PUT is **idempotent** and represents a *full overwrite*.

- Every record attribute must be reassigned based on payload. If an optional field is missing in payload, it **MUST** be reset to its default (e.g., None).
- Uses Unit-of-Work dirty tracking: altering loaded model properties marks attributes as dirty, automatically generating minimal SQL UPDATE on `commit()`.

PUT IMPLEMENTATION – FULL OVERWRITE PATTERN

```
async def replace_user(user_id: int, payload: UserPutSchema, session: AsyncSession):
    user = await session.get(User, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")

    # Full Overwrite: Replace ALL fields (Explicitly set optional missing values to None)
    user.username = payload.username
    user.email = payload.email
    user.bio = payload.bio # Overwrites existing bio even if None
    user.is_active = payload.is_active

    await session.commit()
    await session.refresh(user)
    return user
```

4. HTTP PATCH → PARTIAL MODIFICATION (UPDATE)

PATCH

HOW PATCH WORKS IN SQLALCHEMY 2.0

HTTP PATCH modifies *only specified fields*, leaving all omitted fields untouched.

- **ORM Dynamic Assignment:** Iterate over payload dictionary and assign attributes using `setattr(instance, key, value)`.
- **Direct Bulk Update:** Use `update(User).where(...).values(**patch_data)` to execute direct query without loading entity into session memory first.

PATCH IMPLEMENTATION – DYNAMIC FIELD ASSIGNMENT

```
# --- Pattern A: Dynamic Attribute Assignment on ORM Entity ---
async def patch_user_orm(user_id: int, patch_dict: dict, session: AsyncSession):
    user = await session.get(User, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")

    for key, value in patch_dict.items():
        if hasattr(user, key):
            setattr(user, key, value) # Only modifies provided keys

    await session.commit()
    return user

# --- Pattern B: Direct Core Update SQL Query ---
async def patch_user_direct(user_id: int, patch_dict: dict, session: AsyncSession):
    stmt = update(User).where(User.id == user_id).values(**patch_dict)
    result = await session.execute(stmt)
    await session.commit()
```

5. HTTP DELETE → REMOVE DATA (DELETE)

DELETE

HOW DELETE WORKS IN SQLALCHEMY 2.0

HTTP DELETE removes data permanently or marks soft deletes.

- **ORM Delete:** `session.delete(instance)` marks object for deletion; triggers cascading relationships if configured in ORM models.
- **Direct Query Delete:** `delete(User).where(...)` executes targeted SQL immediately without loading entity into memory.

DELETE IMPLEMENTATION – ORM CASCADES VS DIRECT STATEMENT

```
# --- Pattern A: ORM Instance Delete (Triggers Cascades) ---
async def delete_user(user_id: int, session: AsyncSession):
    user = await session.get(User, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")

    await session.delete(user)
    await session.commit() # Emits SQL DELETE query

# --- Pattern B: Fast Direct Statement ---
async def delete_user_fast(user_id: int, session: AsyncSession):
    stmt = delete(User).where(User.id == user_id)
    result = await session.execute(stmt)
    if result.rowcount == 0:
        raise HTTPException(status_code=404, detail="User not found")
    await session.commit()
```

6. COMPLETE HTTP MAPPING CHEAT-SHEET

HTTP Method	Semantics	SQL Construct	Session Method	Best Used For
GET	Read (Safe, Idempotent)	<code>select(Model)</code>	<code>session.get() / scalars()</code>	Retrieving individual items or collection lists
POST	Create (Non-Idempotent)	<code>insert(Model)</code>	<code>session.add() + commit()</code>	Adding new records and getting DB-generated IDs
PUT	Replace (Idempotent)	Dirty Tracking	Attribute assignment + <code>commit()</code>	Full resource overwrites (all fields supplied)
PATCH	Partial (Non-Idempotent)	<code>update(Model)</code>	<code>setattr()</code> or <code>session.execute()</code>	Updating 1 or 2 fields without replacing others
DELETE	Remove (Idempotent)	<code>delete(Model)</code>	<code>session.delete() + commit()</code>	Deleting records or firing cascade deletes

CRITICAL BEST PRACTICES IN 2.0

1. Scalars vs Execute

`session.execute(select(User))` returns tuples holding objects e.g., `(User, )`. Always use `session.scalars(...)` to get unwrapped User objects directly.

2. Async Session Management

Always wrap routes in a transaction block or issue explicit `await session.commit()`. Uncommitted session changes are lost upon context exit.

3. Flushing vs Committing

Calling `await session.flush()` pushes changes to database buffers without finalizing transaction—ideal for acquiring foreign keys before dependent operations. `commit()` writes data permanently to database.